2. Regular Expressions

Task 1 - Introduction

Regular expressions (or Regex) are patterns of text that you define to search documents and match exactly what you're looking for.

Task 2 - Charsets

A charset is defined by enclosing in `[` square brackets `]` the character(s), or range of characters that you want to match. Then, it finds **every occurrence** of the pattern you have defined in the file/text you are searching.

`[abc]` will match `a`, `b`, and `c` (every occurrence of each letter)

`[abc]zz` will match `azz`, `bzz`, and `czz`.

You can also use a `-` dash to define ranges:

`[a-c]zz` is the same as above.

And then you can combine ranges together:

`[a-cx-z]zz` will match `azz`, `bzz`, `czz`, `xzz`, `yzz`, and `zzz`.

Most notably, this can be used to match any alphabetical character:

`[a-zA-Z]` will match any **single** letter (lowercase or uppercase).

You can use numbers too:

`file[1-3]` will match `file1`, `file2`, and `file3`.

Then, there is a way to **exclude** characters from a charset with the `^` hat symbol, and include everything else.

`[^k]ing` will match `ring`, `sing`, `$ing`, but not `king`.

Of course, you can exclude charsets, not just single characters.

`[^a-c]at` will match `fat` and `hat`, but not `bat` or `cat`.

Answer the questions below

> Match all of the following characters: c, o, g **[cog]**

> Match all of the following words: cat, fat, hat **[cfh]at**

> Match all of the following words: Cat, cat, Hat, hat **[CcHh]at**

> Match all of the following filenames: File1, File2, file3, file4, file5, File7, file9 **[Ff]ile[1-9]**

> Match all of the filenames of question 4, except "File7" (use the hat symbol) **[Ff]ile[^7]**

Task 3 - Wildcards and optional characters

The wildcard that is used to match any single character (except the line break) is the `.` dot. That means that `a.c` will match `aac`, `abc`, `a0c`, `a!c`, and so on.

Also, you can set a character as optional in your pattern using the `?` question mark. That means that `abc?` will match `ab` and `abc`, since the `c` is optional.

Note: If you want to search for `.` a literal dot, you have to **escape it** with a `\` reverse slash. That means that `a.c` will match `a.c`, but also `abc`, `a@c`, and so on. But `a\.c` will match **just** `a.c`.

Answer the questions below

> Match all of the following words: Cat, fat, hat, rat **.at**

> Match all of the following words: Cat, cats **[Cc]ats?**

> Match the following domain name: cat.xyz **cat\.xyz**

> Match all of the following domain names: cat.xyz, cats.xyz, hats.xyz **[ch]ats? \.xyz**

> Match every 4-letter string that doesn't end in any letter from n to z **...[^n-z]**

> Match bat, bats, hat, hats, but not rat or rats (use the hat symbol) **[^r]ats?**

Task 4 - Metacharacters and repetitions

There are easier ways to match bigger charsets. For example, `\d` is used to match any **single** digit. Here's a reference:

`\d` matches a digit, like `9`

`\D` matches a non-digit, like `A` or `@`

`\w` matches an alphanumeric character, like `a` or `3`

`\W` matches a non-alphanumeric character, like `!` or `#`

`\s` matches a whitespace character (spaces, tabs, and line breaks)

`\S` matches everything else (alphanumeric characters and symbols)

Here's a reference for each repetition along with how many times it matches the preceding pattern:

`{12}` - **exactly 12** times.

`{1,5}` - **1 to 5** times.

`{2,}` - **2 or more** times.

- - **0 or more** times.

`+` - **1 or more** times.

Answer the questions below

> Match the following word: catssss **cats{4}**

> Match all of the following words (use the * sign): Cat, cats, catsss **[Cc]ats***

> Match all of the following sentences (use the + sign): regex go br, regex go brrrrrr **regex go br+**

> Match all of the following filenames: ab0001, bb0000, abc1000, cba0110, c0000 (don't use a metacharacter) **[abc]{1,3}[01]{4}**

> Match all of the following filenames: File01, File2, file12, File20, File99 **[Ff]ile\d{1,2}**

> Match all of the following folder names: kali tools, kali tools **kali\s+tools**

> Match all of the following filenames: notes~, stuff@, gtfob#, lmaoo! **\w{5}\W**

> Match the string in quotes (use the * sign and the \s, \S metacharacters): "2f0h@f0j0%! a)K!F49h!FFOK" **\S*\s*\S***

> Match every 9-character string (with letters, numbers, and symbols) that doesn't end in a "!" sign **\S{8}[^!]**

> Match all of these filenames (use the + symbol): .bash_rc, .unnecessarily_long_filename, and note1 **\.?\w+**

Task 5 - Starts with/ ends with, groups, and either/ or

Sometimes it's very useful to specify that we want to search by a certain pattern **in the beginning or the end of a line**. We do that with these characters:

`^` - starts with

`$` - ends with

So for example, if you want to search for a line that **starts with** `abc`, you can use `^abc`.

If you want to search for a line that **ends with** `xyz`, you can use `xyz$`.

Note: The `^` hat symbol is used to exclude a charset when enclosed in `[` square brackets `]`, but when it is not, it is used to specify the beginning of a word.

You can also define groups by enclosing a pattern in `(` parentheses `)`. This function can be used for many ways that are not in the scope of this tutorial. We will use it to define an **either/ or** pattern, and also to repeat patterns. To say "or" in Regex, we use the `|` pipe.

For an "either/or" pattern example, the pattern `during the (day|night)` will match both of these sentences: `during the day` and `during the night`.

For a repetition example, the pattern `(no){5}` will match the sentence `nononononо`.

Answer the questions below

> Match every string that starts with "Password:" followed by any 10 characters excluding "0", irrespective of the position. **Password:[^0]{10}**

> Match "username: " in the beginning of a line (note the space!) **^username:\s**

> Match every line that doesn't start with a digit (use a metacharacter) **^\D**

> Match this string at the end of a line: EOF$ **EOF\$$**

> Match all of the following sentences: I use nano I use vim **I use (nano|vim)**

> Match all lines that start with , $followedbyanysingledigit$, $followedby$, followed by one or more non-whitespace characters **\$\d\$\S+**

> Match every possible IPv4 IP address (use metacharacters and groups) **(\d{1,3}\.){3}\d{1,3}**

> Match all of these emails while also adding the username and the domain name (not the TLD) in separate groups (use \w): hello@tryhackme.com, username@domain.com, dummy_email@xyz.com **(\w+)@(\w+)\.com**

Task 6 - Conclusion

With regex, you have to think specific, but not **too** specific, because then you might come up with complicated solutions when there are other more elegant and simple ones.